main.c

Share Run

```c
#include <stdio.h>
int linearSearch(int arr[], int n, int key) {
    for(int i = 0; i < n; i++) {
        if(arr[i] == key)
            return i;
    }
    return -1;
}
int main() {
    int arr[] = {3, 5, 7, 9};
    int n = sizeof(arr) / sizeof(arr[0]);
    int key = 7;

    int index = linearSearch(arr, n, key);

    if(index != -1)
        printf("Found at index: %d\n", index);
    else
        printf("Not found\n");
    return 0;
}
```

Output

```
Found at index: 2


=== Code Execution Successful ===
```

main.c

Share Run

```c
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    for (int i = 0; i < 5; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

Output

```
10 20 30 40 50

=== Code Execution Successful ===
```

main.c

Share Run

```c
#include <stdio.h>
int binarySearch(int arr[], int n, int key){
    int low = 0, high = n - 1;
    while (low <= high)   {
        int mid = (low + high) / 2;
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;  }
    return -1; }
int main(){
    int arr[] = {1, 3, 5, 7, 9};
    int n = sizeof(arr) / sizeof(arr[0]);
    int key = 7;
    int index = binarySearch(arr, n, key);
    if (index != -1)
        printf("Found at index %d\n", index);
    else
        printf("Not found\n");
    return 0:
```

Output

```
Found at index 3


=== Code Execution Successful ===
```

main.c

Share Run

```c
#include <stdio.h>
#define MAX 5

int stack[MAX];
int top = -1;

void push(int data) {
    if (top == MAX - 1) {
        printf("Stack Overflow");
        return;  // stop here
    } else {
        stack[++top] = data;  // clearer increment order
    }
}

int main() {
    push(10);
    push(20);
    printf("%d", stack[top]); // prints 20
    return 0;
}
```

Output

```
20

=== Code Execution Successful ===
```

main.c

Share Run

Output

```c
#include <stdio.h>
#define MAX 5
int stack[MAX];
int top = 2;
int pop() {
    if (top == -1)
        return -1;
    return stack[top--];
}
int main() {
    // example values
    stack[0] = 10;
    stack[1] = 20;
    stack[2] = 30;
    int x = pop();
    printf("%d\n", x);
    return 0;
}
```

```
30


=== Code Execution Successful ===
```

main.c

Share Run

```c
#include <stdio.h>
#define MAX 100
int stack[MAX];
int top = 0;  // if top==0 means stack is empty
int pop() {
    if (top == 0) {
        printf("Underflow\n");
        return -1;
    }
    int val = stack[top - 1];
    top--;
    return val;
}
int main() {
    stack[top++] = 10;
    stack[top++] = 20;
    printf("%d\n", pop());
    printf("%d\n", pop());
    printf("%d\n", pop());

    return 0;
}
```

Output

```
20
10
Underflow
-1


=== Code Execution Successful ===
```

main.c

Share Run

```c
#include <stdio.h>
#define MAX 5
void enqueue(int queue[], int *rear, int x) {
    if (*rear == MAX) {
        printf("Overflow\n");
        return;
    } else {
        queue[*rear] = x;
        (*rear)++;
        printf("Value is %d\n", x);
    }
}
int main() {
    int queue[MAX], x;
    int front = 0, rear = 0;  // front is unused in enqueue-only code
    printf("Enter the value: ");
    scanf("%d", &x);
    enqueue(queue, &rear, x);
    return 0;
}
```

Output

```
Enter the value: 12
Value is 12


=== Code Execution Successful ===
```

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *next;
};
int main() {
    struct Node *newNode;

    newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }
    newNode->data = 10;
    newNode->next = NULL;
    printf("%d", newNode->data);
    free(newNode);
    return 0;
}
```

Output

```
10

=== Code Execution Successful ===
```

main.c

Share Run

```c
#include <stdio.h>
#include <stddef.h>
struct Node {
    int data;
    struct Node *next;
};
void display(struct Node *head)
{
    while (head != NULL)
    {
        printf("%d ", head->data);
        head = head->next;
    }
}
int main(void)
{
    struct Node a = {1, NULL};
    display(&a);
    return 0;
}
```

Output

```
1

=== Code Execution Successful ===
```

main.c

Share Run

```c
#include <stdio.h>
#define SIZE 5
int queue[SIZE];
int front = -1, rear = -1;
void enqueue(int item){
    if (front != -1 && ((rear + 1) % SIZE == front)){
        printf("Queue Full\n");
        return;}
    if (front == -1)
        front = 0;
    rear = (rear + 1) % SIZE;
    queue[rear] = item;}
void display(){
    if (front == -1){
        printf("Queue Empty\n");
        return;}
    printf("Queue elements: ");
    int i = front;
    while (1) {
        printf("%d ", queue[i]);
        if (i == rear)
            break:
```

Output

```
Queue elements: 10 20 30


=== Code Execution Successful ===
```